# LiteVNA

## User Guide

Based on the v1.3 firmware

# 5.Appendix– USB data interface

To ensure compatibility with current PC software, LiteVNA uses the same data communication protocol as SAA2, appears as a USB CDC (Communications Device Class) virtual serial port both during normal operation and in DFU mode. PC software can issue commands and request data by sending and receiving data on the virtual serial port. Communications protocol is identical in the two cases, with only register layouts differing.

## Protocol description

Only the host may initiate commands by sending one or more bytes on the virtual serial port. Each command may have a different length. There is no separator delimiting each command. The device may not send data to the host except for replies to a host-to-device command.

The following table lists all supported commands and their byte encodings, and is applicable both during normal operation and in DFU mode.

# Host to device command list

All byte values in the table are in hexadecimal. B0 to B5 denote bytes 0 to 5. B0 is the opcode.

| B0 | B1 | B2 | B3 | B4 | B5 | Name | Description |
|---|---|---|---|---|---|---|---|
| 00 | - | - | - | - | - | NOP | No operation |
| 0d | - | - | - | - | - | INDICATE | Device always replies with ascii ‘2’ (0x32) |
| 10 | (AA) | - | - | - | - | READ | Read a 1-byte register at address AA.<br>Reply is one byte, the read value. |
| 11 | (AA) | - | - | - | - | READ2 | Read a 2-byte register at address AA.<br>Reply is 2 bytes, the read value. |
| 12 | (AA) | - | - | - | - | READ4 | Read a 4-byte register at address AA.<br>Reply is 4 bytes, the read value. |
| 18 | (AA) | (NN) | - | - | - | READFIFO | Read NN values from a FIFO at address AA.<br>Reply is the read values in order.<br>Each value can be more than one byte and is determined by the FIFO being read. |
| 20 | (AA) | (XX) | - | - | - | WRITE | Write XX to a 1-byte register at address AA.<br>There is no reply. |
| 21 | (AA) | (X0) | (X1) | - | - | WRITE2 | Write X0 to AA, then X1 to AA+1.<br>There is no reply. |
| 22 | (AA) | (X0) | (X1) | (X2) | (X3) | WRITE4 | Write X0..X3 to registers starting at AA. There is no reply. |
| 23 | (AA) | (X0) | (X1) | (X2) | ... | WRITE8 | This command is 10 bytes in total.<br>Bytes 2..9 correspond to X0..X7.<br>Write X0..X7 to registers starting at AA.<br>There is no reply. |
| 28 | (AA) | (NN) | … | | | WRITEFIFO | Write NN bytes into a FIFO at address AA.<br>NN bytes of data to be written into the FIFO should follow “NN”.<br>There is no reply. |

# Register descriptions

The following table lists all registers accessible during normal operation.
All addresses are in hexadecimal.
Multi-byte integer registers are encoded in Little Endian. Lowest numbered registers contain the least significant portions of the integer.

| Address | Name | Description |
|---|---|---|
| 00..07 | sweepStartHz | Sets the sweep start frequency in Hz. **uint64**. |
| 10..17 | sweepStepHz | Sets the sweep step frequency in Hz. **uint64**. |
| 20..21 | sweepPoints | Sets the number of sweep frequency points. **uint16**. |
| 22..23 | valuesPerFrequency | Sets the number of data points to output for each frequency. **uint16**. |
| 26 | rawSamplesMode | Writing 1 switches USB data format to raw samples mode and leaves this protocol. |
| 30 | valuesFIFO | Returns VNA sweep data points. Each value is 32 bytes.<br>Writing any value (using **WRITE** command) clears the FIFO.<br>See FIFO data format section below. |
| 40 | Average | Set the average multiplier (FFT sample point multiplier, which actually changes the IFBW). Allowable range 1-80 (0x01-0x50). |
| 41 | LowFrequencyPower | Set the power of the low frequency signal generator (MS5351), default value 0x01, setting range 0x01-0x03. |
| 42 | HighFrequencyPower | Set the power of the High frequency signal generator (MAX2871), default value 0x03, setting range 0x01-0x03. |
| 44 | ChannelSelect | Select the channel to scan, single channel scanning is faster, the closed channel returns invalid values.<br>0x01 S11, 0x02 S21, 0x00 S11 and S21 |
| 58 | UnixTime | Set the system time for liteVNA. Use 4 bytes to send Unix Timestamp. |
| f0 | deviceVariant | The type of device this is. Always 0x02 for liteVNA. |
| f1 | protocolVersion | Version of this wire protocol. Always 0x01. |
| f2 | hardwareRevision | Hardware revision. |
| f3 | firmwareMajor | Firmware major version. |
| f4 | firmwareMinor | Firmware minor version. |

**Remarks sweepStartHz**, **sweepStepHz**, and **sweepPoints** together set the sweep parameters of the VNA.

Writing any value to these registers will immediately terminate the on-device UI and put the device in “USB data mode”, where the PC has full control over VNA operation.

You can not observe user entered sweep parameters (from the device UI) by reading these registers.

Sweep is always running and can not be paused.

## FIFO data format

The values read from **valuesFIFO** are 32 bytes each. The following table lists the fields in each value. All byte offsets are in hexadecimal. All multi-byte integers are encoded in Little Endian. Lowest numbered bytes contain the least significant portions of the integer.

| Bytes | Name | Description | Type |
|---|---|---|---|
| 00..03 | fwd0Re | Real part of channel 0 outgoing wave. | int32 |
| 04..07 | fwd0Im | Imaginary part of channel 0 outgoing wave. | int32 |
| 08..0b | rev0Re | Real part of channel 0 incoming wave. | int32 |
| 0c..0f | rev0Im | Imaginary part of channel 0 incoming wave. | int32 |
| 10..13 | rev1Re | Real part of channel 1 incoming wave. | int32 |
| 14..17 | rev1Im | Imaginary part of channel 1 incoming wave. | int32 |
| 18..19 | freqIndex | Frequency index, 0 to sweepPoints - 1. | uint16 |
| 1a..1f | (reserved) | (reserved) | - |

**valuesFIFO** is continuously being filled with new sweep data regardless of whether it is being read.
If you wish to do on-demand sweeps, it is necessary to clear stale data before reading from the FIFO. The FIFO can be cleared by writing (with the **WRITE** command) any value to the FIFO address.

**valuesFIFO** returns raw values representing the in-phase and quadrature part of the measured waves, which never have user calibration applied. You can not access the on-device user calibrations or calibrated data over USB.

**fwd0Re/fwd0Im** is referred to as the reference channel. All complex values read from **valuesFIFO** can be at a random phase, so you must divide (using complex division) each value by the reference channel to get absolute phase and magnitude values.

# Register descriptions (DFU mode)

The following table lists all registers accessible in DFU mode.

All addresses are in hexadecimal.

Multi-byte integer registers are encoded in Little Endian. Lowest numbered registers contain the least significant portions of the integer.

| Address | Name | Description |
|---|---|---|
| e0..e3 | flashWriteStart | Current flash write address. **uint32**.<br>Set this to the address to start writing at. |
| e4 | flashFIFO | Writing to this FIFO will write data into flash starting at **flashWriteStart**. **flashWriteStart** will be incremented by the number of bytes written. |
| e8..eb | userArgument | The user argument to pass to the program upon soft reset. **uint32**. |
| ef | doReboot | Write 0x5e to initiate a soft reset. |
| f0 | deviceVariant | The type of device this is. Always 0x02 for LiteVNA. |
| f1 | protocolVersion | Version of this wire protocol. Always 0x01. |
| f2 | hardwareRevision | Hardware revision. Always 0x00 in DFU mode. |
| f3 | firmwareMajor | Firmware major version. Always 0xff in DFU mode. |
| f4 | firmwareMinor | Firmware minor version (of the bootloader). |

## Writing to flash

The procedure to write a new firmware image to flash are as follows.

1. Connect the device to the PC over USB and put the device into DFU mode.
2. Open the virtual serial port in raw mode (platform specific).
3. Write the address you wish to start flashing from to **flashWriteStart**.
4. Use the **WRITEFIFO** command to send up to 255 bytes at a time to **flashFIFO**.
   Each **WRITEFIFO** command can be followed with a **INDICATE** command, which will reply with ‘2’ only after the flash operation is complete.
   There is no flow control on the virtual serial port and you must limit the amount of outstanding writes to no more than 2048 bytes.
5. (Optional) Write 0x5e to **doReboot** to soft reset the device.